

Seven Question Types, Seven Frameworks, and How to Compress Each into a Headline

Answer-first reasoning for technical interviews, built on the Minto Pyramid

Harry Truong

September 2026

Purpose

This document helps you recognize a question's type quickly, pick the reasoning structure that fits it, and put one short governing claim at the top of the answer. Everything else in the framework exists only to prove, explain, or make that claim concrete.

Core conclusion

A framework is scaffolding for practice and a recovery mechanism when you get stuck. In the interview, the goal is to produce the headline first and unfold the reasoning behind it.

Mental model

Signal	Operation in your head	What you say out loud
The question	The router picks the type	
The question type	The framework organizes the reasoning	
Reasoning is sufficient	Compress it into a governing claim	The headline, at the top of the answer
The headline	Expand it along the framework	Evidence, mechanism, trade-off, or story

The map of seven question types

The router does not need to classify every word perfectly. Look for the main purpose of the question: to define, to explain, to design, to diagnose, to optimize, to choose, or to prove judgment through experience.

Type	Router	Framework	Headline shape	Opening pattern
Define	WHAT IS X?	DMEI	Essence, taken directly from D written as CDM	X is essentially...
Explain	HOW OR WHY X?	IMO	Causal claim	X works because...
Design	HOW BUILD X?	GFM	Design principle	The key design principle is...
Diagnose	WHY FAILING?	HEFR	Problem frame + diagnostic direction	I would treat this as... and locate...
Optimize	HOW IMPROVE?	OCBED	Problem frame + optimization direction	I would optimize... while...
Compare or Choose	X VS Y?	CTD	Conditional decision rule	Choose X when...; otherwise...
Experience	YOUR STORY?	STAR+L	Generalized judgment or takeaway	When..., I...; this taught me...

Routing a question that looks hybrid

- Pick the intent the interviewer wants to evaluate, not the surface verb. “How would you debug...” is Diagnose, not Design.
- If a question both designs and compares, use the primary framework for the decision and borrow a secondary one for support. Example: GFM sets the architecture; CTD explains the technology choice.
- If you are unsure, ask yourself: “Is the best output of this answer a definition, a cause, a design, a diagnosis, an optimization, a decision, or evidence of judgment?”

Timing rules

Time budget	Practical structure
30 seconds	Headline + 2 supporting points
60–90 seconds	Headline + a condensed framework + implication or result
Deep dive	Open one branch at a time as the interviewer asks; do not dump the whole checklist up front

The Minto Pyramid and compression

In this document, the top of the pyramid and the headline are the same thing: one short concluding sentence placed at the very top. “Top” is how you encode it in your head; “headline” is the sentence

you actually say.

A good headline is the shortest sentence that still preserves the core logic of the whole framework and can regenerate the support beneath it.

Three standards for a headline

- **Answer-first:** it answers exactly what the interviewer asked, without opening on context or a checklist.
- **Governing:** every point below it explains, proves, or elaborates the headline.
- **Generative:** from the headline you can reconstruct the main reasoning. If the sentence is too generic, it has not compressed enough information.

Not “pick one box”, but “compress a relation”

One letter of a framework can be the pivot you remember, but the headline is not confined to that box. OCBED pivots on B, the Bottleneck, yet a strong headline must join Objective + Constraint + Bottleneck into one strategy. IMO pivots on M, the Mechanism, yet the headline must explain why the mechanism turns the input into the output.

Framework	Correct compression	Avoid
DMEI	D written as CDM already is the essence; use it directly	Repeating a vague dictionary definition
IMO	I + M + O → a causal claim	Narrating the mechanism without linking it to the effect
GFM	G + F + M → a design principle	Restating the obvious goal or listing controls
HEFR	Symptom + H/E → a problem frame and an isolation direction	Guessing the root cause before there is evidence
OCBED	O + C + B → an optimization direction	Listing optimization tricks
CTD	C + T + D → a conditional decision rule	An absolute conclusion that X beats Y
STAR+L	Story → a generalized judgment	Telling the long story, then reaching the lesson

Self-check questions

- If you could say only one sentence, which sentence answers best and still explains the rest?
- Does the headline contain an insight the interviewer did not already have, or does it just restate the question?
- Do the supporting points all prove one claim, or are they a loose list?

The seven frameworks

1. DMEI

Define | Router: WHAT IS X? | Definition → Mechanism → Example → Implication

DMEI explains a concept from its essence to how it operates to what it means in practice. D is not a generic definition; D is written as CDM.

- **D = CDM.** Category: what kind of thing X is. Distinctions: where it differs from the things near it. Meaning: what that difference makes possible.
- **M:** the core mechanism or flow.
- **E:** one concrete example, kept short.
- **I:** why the concept matters, or its practical consequence.

How to compress it into a headline. D written as CDM already is the essence and the top of the pyramid. M, E, and I are support; no separate compression function is needed.

Spoken pattern: *X is a [category] that [distinction], enabling [meaning].*

Question: What is an agent harness in an agentic AI system?

An agent harness is the orchestration system around an LLM that manages the agent loop and turns model decisions into controlled execution.

- **D / CDM:** an orchestration system; it surrounds the LLM rather than being the LLM; it makes a controlled, reliable workflow possible.
- **M:** build context → the LLM decides → validate and execute the tool → return the observation → loop until a stop condition.
- **E:** a scheduling request is turned into a tool call, the result is fed back to the model, and the loop ends with a success message.
- **I:** the model no longer just generates text; it can run an autonomous, stateful, controlled workflow.

2. IMO

Explain | Router: HOW OR WHY X? | Input → Mechanism → Output

IMO makes a causal chain explicit: what the input state is, which mechanism acts on it, and how the output changes.

- **I:** the input or initial state.
- **M:** the transformation or mechanism that produces the change.
- **O:** the output or effect that needs explaining.

How to compress it into a headline. Compress I + M + O into one causal claim: why O appears when M acts on I. The headline is not a transcript of M.

Spoken pattern: *X improves Y because [mechanism linking input to output].*

Question: Why does reranking improve retrieval quality in a RAG system?

Reranking improves retrieval because it separates fast candidate recall from more precise relevance scoring.

- **I:** the retriever returns a candidate set with decent recall but inaccurate ordering.

- **M:** the reranker applies a more precise query–document comparison on that small set.
- **O:** relevant evidence moves to the top and the LLM receives better context.

IMO through four short examples

Topic	Causal headline	Compressed logic
Hybrid search	Hybrid search improves retrieval because lexical and semantic signals cover complementary failure modes.	exact match + semantic similarity → more robust retrieval
Reranking	Reranking separates broad candidate recall from precise relevance scoring.	fast first stage + precise second stage → better ranking
Caching	Caching reduces latency by replacing repeated expensive computation or I/O with a much cheaper lookup.	recomputation → cache hit → less critical-path work
Batch normalization	Batch normalization makes training easier by making optimization less sensitive to changing activation scales.	normalize + learned scale/shift → more stable updates

3. GFM

Design | Router: HOW BUILD X? | Goal → Failure modes → Mechanisms

GFM designs from the goal and its failure modes instead of jumping straight to components. Every mechanism must trace back to a failure that matters.

- **G:** the goal or invariant the system must guarantee.
- **F:** the ways the goal can be broken.
- **M:** the mechanisms that handle each failure mode.

How to compress it into a headline. Synthesize G + F + M into one design principle that organizes the whole architecture. Add a problem frame if it makes the principle clearer.

Spoken pattern: *The key design principle is to [architectural principle].*

Question: How would you design an AI agent that can safely execute actions on behalf of a user?

The key is to separate agent reasoning from controlled action execution.

- **G:** keep useful autonomy while preventing unauthorized, repeated, or irreversible actions.
- **F:** wrong permissions, wrong arguments, missing approval, retries that double side effects, crashes mid-action, missing audit.
- **M:** scoped permissions, validation, risk-based approval, idempotency, durable state, and an audit trail.
- **Architecture:** the LLM proposes an action; the execution layer decides whether it is allowed and how it runs.

4. HEFR

Diagnose | Router: WHY FAILING? | Hypotheses → Evidence → Fix → Re-evaluate

HEFR prevents jumping straight to a fix. You form competing hypotheses, use evidence to tell them apart, fix the confirmed cause, and verify again.

- **H:** the plausible causes that could explain the symptom.
- **E:** evidence or tests with high discriminating power.
- **F:** fix the confirmed source of the failure.
- **R:** replay, regression, and monitoring to confirm recovery.

How to compress it into a headline. Compress the symptom together with H/E into a problem frame and a diagnostic direction. Until the evidence is sufficient, do not turn a hypothesis into a firm conclusion.

Spoken pattern: *I would treat this as [problem frame] and [diagnostic direction].*

Question: Production RAG quality drops, but offline evaluation remains normal. How would you investigate?

I would treat this as a production–offline divergence problem and locate the first point where the two pipelines behave differently.

- **H:** query shift, stale index, filtering, version mismatch, different preprocessing, timeouts or fallbacks.
- **E:** replay the same queries and compare stage by stage: preprocessing, retrieval, reranking, context, model, and output.
- **F/R:** fix the first point of divergence; replay the failing cases, run regression, and monitor production.

5. OCBED

Optimize | Router: HOW IMPROVE? | Objective → Constraints → Bottleneck → Experiments → Decision

OCBED turns the answer from a list of techniques into an optimization process driven by evidence and constraints.

- **O:** the metric to improve.
- **C:** the quality bar or limits that must not be violated.
- **B:** the component that currently dominates the objective.
- **E:** experiments aimed at that bottleneck.
- **D:** choose the change with the best trade-off.

How to compress it into a headline. Compress O + C + B into a problem frame and an optimization direction; E and D support how you execute and decide.

Spoken pattern: *I would optimize [dominant bottleneck] while treating [constraint] as a hard constraint.*

Question: RAG quality is good, but p95 latency is too high. How would you reduce latency without hurting quality?

I would optimize the dominant latency contributor rather than the whole pipeline, while treating answer quality as a hard constraint.

- **O/C:** reduce p95 while preserving quality and reliability.
- **B:** profile retrieval, reranking, context construction, TTFT, generation, and external calls.
- **E:** try caching, context trimming, model routing, candidate depth, or parallelization only at the measured bottleneck.
- **D:** keep the change that reduces p95 the most without crossing the quality threshold.

6. CTD

Compare or Choose | Router: X VS Y? | Criteria → Trade-offs → Decision

CTD avoids absolute answers. Pick the criteria that actually decide, compare the options on those criteria, and state the conditions under which you choose each.

- **C:** the criteria that truly decide.
- **T:** how the options differ and what each trades away.
- **D:** the decision, conditioned on context.

How to compress it into a headline. Compress C + T + D into one conditional decision rule: choose X when A matters more than B; choose Y when the condition flips.

Spoken pattern: *Choose X when A matters more than B; otherwise choose Y.*

Question: When would you choose a managed model API over self-hosting an open-source model?

I would choose a managed API when operational simplicity and speed to market matter more than deep control over the model stack.

- **C:** time to market, ops burden, scale, cost, latency, privacy, customization, and control.
- **T:** managed removes serving, GPU, and upgrade work but gives less control and can cost more at scale; self-hosting is the reverse.
- **D:** self-host when data constraints, customization, control, or scale economics become the dominating factor.

7. STAR+L

Experience | Router: YOUR STORY? | Situation → Task → Action → Result → Learning

STAR+L turns a story into evidence of how you think and act. S and T are only enough to understand the setting; A and R prove judgment; L generalizes what was learned.

- **S:** the minimal context.
- **T:** your own responsibility or decision.
- **A:** the actions and their rationale.
- **R:** the observable result.
- **L:** the learning, raised into a reusable judgment.

How to compress it into a headline. Do not copy the raw Learning. Compress the whole story into the generalized judgment or takeaway that the story proves.

Spoken pattern: *When [condition], I [judgment rule]. I learned this when...*

Question: Tell me about a time you had to make a technical decision with incomplete information.

When information is incomplete, I commit strongly only where requirements are stable and preserve flexibility where uncertainty remains high.

- **S/T:** a digital-twin AI agent; production integration, streaming, and deployment were still changing; I had to choose the level of prototyping and the production structure.
- **A:** separated the relatively stable agent logic from the uncertain integration and infrastructure.
- **R:** the team kept iterating the core agent while changing the surrounding system with less rework.
- **L / Judgment:** distinguish reversible from irreversible decisions; commit in proportion to how stable the assumptions are.

A practice workflow, all the way to reflex

The goal is not to memorize more acronyms. The goal is to compile the frameworks into a mental schema, so that hearing the question produces the governing claim.

Stage	Flow	Exit criterion
1. Deliberate practice	Question → classify → framework → fill → compress → headline	You produce a correct headline and can explain why it governs the support
2. Assisted	Question → suggested framework → headline → expand	You no longer need to fill every box before saying the headline
3. Retrieval	Question → headline → expand; the framework stays in the background	You produce a top-level sentence within seconds
4. Interview	Question → answer	Natural, conclusion-first; the framework appears only for recovery

The standard drill for each question

- Classify the question and say the router out loud.
- Fill the framework with keywords, not essays.
- Write a single headline. Check whether the reasoning can be reconstructed from it.
- Answer the 30-second version, then the 60–90-second version.
- Drop the framework and answer naturally; compare whether the claim is still clear and the support still coherent.
- Review with spaced retrieval: look only at the question, produce the headline, then check your notes.

When to use a framework

Mode	Role of the framework
While practicing	A training scaffold: it forces reasoning that is sufficient, structured, and checkable.
When answering well	Latent structure: you do not read the acronym or enumerate the boxes; you say the headline and expand naturally.
When stuck	A fallback: recall the framework and ask the pivot question, whether mechanism, failure, divergence, bottleneck, criteria, or judgment.

Runtime target: Question → compressed claim → supporting reasoning.

Signs you have practiced enough

- You can say the headline in one sentence without enumerating the framework first.
- When probed, each supporting point opens exactly one branch of the framework, not a random list.
- You can change the example or the domain and still keep the right kind of governing claim.
- You can shrink the answer to 30 seconds or grow it to 2 minutes without changing the logic.

One-page cheat sheet

Cue	Framework	Ask yourself	Headline to produce
WHAT IS X?	DMEI	What kind of thing is X, how does it differ, and what does the difference mean?	Essence, taken directly from D = CDM
HOW OR WHY?	IMO	Which mechanism makes the output follow the input?	Causal claim
HOW BUILD?	GFM	Which principle achieves the goal despite the failure modes?	Design principle
WHY FAILING?	HEFR	Which frame makes the direction for isolating the root cause obvious?	Problem frame + diagnostic direction
HOW IMPROVE?	OCBED	Which bottleneck dominates the objective under the constraint?	Problem frame + optimization direction

Cue	Framework	Ask yourself	Headline to produce
X VS Y?	CTD	Which condition flips the choice?	Decision rule
YOUR STORY?	STAR+L	Which judgment does this story prove?	Generalized judgment or takeaway

Seven sample headlines

- **Define:** An agent harness is the orchestration system around an LLM that manages the agent loop and turns model decisions into controlled execution.
- **Explain:** Reranking improves retrieval because it separates fast candidate recall from more precise relevance scoring.
- **Design:** The key is to separate agent reasoning from controlled action execution.
- **Diagnose:** I would treat this as a production–offline divergence problem and locate the first point where the two pipelines behave differently.
- **Optimize:** I would optimize the dominant latency contributor while treating answer quality as a hard constraint.
- **Choose:** I would choose a managed API when operational simplicity and speed to market matter more than deep control.
- **Experience:** When information is incomplete, I commit strongly only where requirements are stable and preserve flexibility elsewhere.

The final rule

Do not try to say the framework. Use the framework to find the most important sentence, say that sentence first, and let everything else prove it.

Companion posts, one per framework, live at <https://harrytruong0804.github.io/posts/seven-answer-frameworks/>